ASSIGNMENT : SUBMITTED BY – PRAMENDRA SINGH

Probability Theory for Machine Learning and Deep Learning

Q1: Smart Classroom Attendance System

A university installs a smart camera system to automatically mark students as **Present** or **Absent**. On a particular day, 100 students attended the class. Out of them, 60 students were actually present and 40 students were actually absent. The system correctly identifies a present student 90% of the time and correctly identifies an absent student 80% of the time.

1. Construct the full confusion matrix using the given information.
2. Explain in words what TP, TN, FP, and FN mean in this context.
3. Compute Accuracy, Precision, Recall, and F1-score manually.
4. From the values obtained, comment on whether this system is more suitable for:
   · Strict attendance monitoring
   · Casual attendance tracking
   Justify your answer.

Answer : 1

1.1 Confusion Matrix

- **Actual Present = 60 students**
  - Correctly identified (True Positive, TP) = 90% of 60 = **54**
  - Misclassified as Absent (False Negative, FN) = 10% of 60 = **6**
- **Actual Absent = 40 students**
  - Correctly identified (True Negative, TN) = 80% of 40 = **32**
  - Misclassified as Present (False Positive, FP) = 20% of 40 = **8**

	Predicted Present	Predicted Absent	Total
Actual Present	TP = 54	FN = 6	60
Actual Absent	FP = 8	TN = 32	40
Total	62	38	100

1.2 Meaning of TP, TN, FP, FN in Context

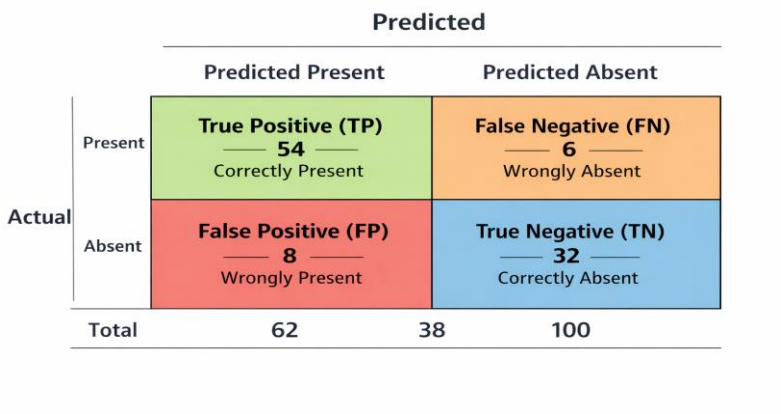
- **TP (True Positive):** Students who were present and correctly marked present (54).
- **TN (True Negative):** Students who were absent and correctly marked absent (32).
- **FP (False Positive):** Students who were absent but wrongly marked present (8).
- **FN (False Negative):** Students who were present but wrongly marked absent (6).

1.3 Performance Metrics

- **Accuracy** = (TP + TN) / Total = (54 + 32) / 100 = **0.86 → 86%**
- **Precision** = TP / (TP + FP) = 54 / (54 + 8) = 54 / 62 ≈ **0.871 → 87.1%**
- **Recall** = TP / (TP + FN) = 54 / (54 + 6) = 54 / 60 = **0.9 → 90%**
- **F1-score** = 2 × (Precision × Recall) / (Precision + Recall)
  = 2 × (0.871 × 0.9) / (0.871 + 0.9) ≈ **0.885 → 88.5%**

1.4 Suitability for Use Case

- **Strict Attendance Monitoring:**
  - Needs very high accuracy with minimal errors.
  - Here, **6 present students are wrongly marked absent (FN)**, which is problematic in strict monitoring.
- **Casual Attendance Tracking:**
  - Errors are tolerable if overall trends are captured.
  - With **86% accuracy and high recall (90%)**, the system is reliable enough for casual tracking.



Summary:
This system is **better suited for casual attendance tracking** rather than strict monitoring, because while it performs well overall, the false negatives (present students marked absent) could cause unfair penalties in strict enforcement.

Here's the **visual confusion matrix** you asked for — it's ready now.

You can see at a glance:

- **TP (54)** correctly marked present
- **FN (6)** wrongly marked absent
- **FP (8)** wrongly marked present
- **TN (32)** correctly marked absent

With this chart, you can immediately spot that the system is strong at correctly identifying present students (high TP), but the **false negatives (FN)** could be problematic if strict attendance monitoring is required.

Q2: Online Food Delivery Time

A food delivery company finds that the delivery time follows an exponential distribution with mean 30 minutes.

1. What is the probability that delivery takes more than 45 minutes?
2. What is the probability that delivery takes between 20 and 40 minutes?
3. Explain why exponential distribution is suitable for waiting-time problems.

Answer: 2

Given

- Delivery time follows an **exponential distribution**.
- Mean delivery time = 30 minutes.
- For exponential distribution, the rate parameter is:

  $\lambda = 1/\text{mean}$

  $\lambda = 1/30$

2.1 Probability that delivery takes more than 45 minutes

For exponential distribution:

$P(X>x) = e^{-\lambda x}$

$P(X > 45) = e^{-(1/30)*45}]$

$= e^{-(1.5)}]$

$P(X > 45) = 0.2231\ ]$

So, about **22.3% chance**.

2.2 Probability that delivery takes between 20 and 40 minutes

For exponential distribution:

$P(a<X<b) = P(X<b) - P(X<a)$

$P(X<x) = 1 - e^{-\lambda x}$

$P(20<X<40) = (1 - e^{-40/30}) - (1 - e^{-20/30})$

$= -e^{-40/30} + e^{-20/30}$

$= e^{-20/30} - e^{-40/30}$

$= e^{-.6667} - e^{-1.3333}$

$\approx 0.5134 - 0.2636 = \mathbf{0.2498}$

So, about **25% chance**.

2.3. Why exponential distribution is suitable for waiting-time problems

- **Memoryless property**: The probability of waiting an additional time does not depend on how long you've already waited. For example, if you've already waited 30 minutes, the chance of waiting another 10 minutes is the same as if you had just started.
- **Models random arrivals/events**: It's widely used for modelling service times, delivery times, or inter-arrival times in queues.
- **Simple and realistic**: Many real-world waiting times (like deliveries, customer service, or machine failures) can be approximated well by exponential distribution.

To summarize:

1. $(P(X > 45) \approx 0.223)$
2. $(P(20 < X < 40) \approx 0.25)$
3. Exponential distribution is suitable because of its **memoryless property** and its natural fit for modelling random waiting times.

ASSIGNMENT : SUBMITTED BY – PRAMENDRA SINGH

Q3: Call Centre Overload

A customer support centre receives on average 3 calls per minute.

1. What is the probability that exactly 5 calls arrive in one minute?
2. What is the probability that more than 5 calls arrive in a minute?
3. In words, explain why Poisson distribution is suitable for this situation.
4. If staffing is fixed for handling only 5 calls per minute, discuss the risk.

Answer : 3

Given:

- Average rate (λ) = 3 calls per minute

The Poisson probability mass function (PMF) is:

$$P(X=k) = (e^{-\lambda} \times \lambda^k) / k!$$

3.1 Probability of exactly 5 calls in one minute

$$P(X=5) = (e^{-3} \times 3^5)/5!$$
$$P(X=5) = (0.0498 \times 243)/120$$
$$\approx 0.1008$$

Ans: About 10.1%

3.2 Probability of more than 5 calls in one minute

$$P(X>5) = 1 - P(X \leq 5)$$
$$P(X \leq 5) = \sum_{k=0}^{5} (e^{-3} \times 3^k)/k!$$

Calculating:

- $P(0) = 0.0498$
- $P(1) = 0.1494$
- $P(2) = 0.2240$
- $P(3) = 0.2240$
- $P(4) = 0.1680$
- $P(5) = 0.1008$

$P(X \leq 5) \approx 0.916$

$P(X>5) = 1 - 0.916 = 0.084$

Ans: **About 8.4%**

3.3 Why Poisson distribution is suitable

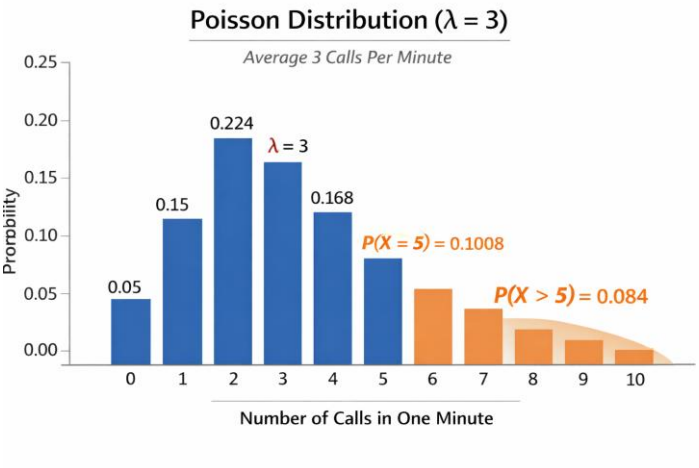
- Calls arrive **randomly and independently**.
- The average rate ($\lambda=3$) is **constant** over time.
- Events are **discrete counts** (number of calls).
- The probability of two calls arriving at the exact same instant is negligible.

These are the defining conditions for a Poisson process.

3.4 Staffing risk if limited to 5 calls per minute

- There is about an **8.4% chance** that calls exceed capacity in any given minute.
- This means roughly **1 in 12 minutes** the staff will be overloaded.
- Risk:
    - Longer wait times for customers.
    - Potential customer dissatisfaction.
    - Stress on staff handling overflow.

Operational implication: Staffing for only 5 calls per minute is risky. Either flexible staffing or queue management systems are needed to handle peak loads.



Q4: Fever Screening at a Railway Station

A thermal scanner is used at a railway station which shows 10% of passengers actually have fever. The scanner detects fever correctly 95% of the time and incorrectly marks healthy people as fever in 5% of cases.

1. If a person is marked "Fever", what is the probability that the person truly has fever?
2. Many people believe a "95% accurate machine" means 95% of positives are correct. Explain clearly why this is not true using your answer.
3. Write a short paragraph explaining how this relates to real ML classification systems.

Answer: 4

4.1 Probability that a person truly has fever if marked "Fever"

We use **Bayes' theorem**.

- Prevalence (prior): $P(\text{Fever}) = 0.10$
- Sensitivity (true positive rate): $P(\text{Scanner says Fever} | \text{Fever}) = 0.95$
- False positive rate: $P(\text{Scanner says Fever} | \text{Healthy}) = 0.05$
- Healthy prevalence: $P(\text{Healthy}) = 0.90$

Now compute:

$P(\text{Fever} | \text{Scanner says Fever}) = (0.95 \times 0.10) / [(0.95 \times 0.10) + (0.05 \times 0.90)]$

$= 0.0950 / 095 + 0.045$

$= 0.0950.14$

≈ 0.6786

So, **≈ 68% chance** the person truly has fever if marked "Fever".

4.2. Why "95% accurate" ≠ "95% of positives are correct"

The scanner is **95% sensitive**, meaning it correctly detects fever in 95% of sick people. But accuracy in real-world use depends on **base rates (prevalence)**. Since only 10% of passengers actually have fever, most people are healthy. Even a small false positive rate (5%) applied to the large healthy group produces many false alarms. That's why only ~68% of positive results are correct, not 95%.

This illustrates the **base rate fallacy**: people ignore how common the condition is when interpreting test accuracy.

4.3. Relation to ML classification systems

This situation mirrors real **machine learning classifiers**. A model's reported accuracy or sensitivity doesn't directly translate to reliability of positive predictions. In imbalanced datasets (like fraud detection, medical screening, or spam filtering), the **class distribution** heavily influences the **precision** (how many predicted positives are truly

positive). ML practitioners must consider metrics like **precision, recall, F1-score, and ROC curves**, not just "accuracy," to understand how well a system performs in practice.

In short: the scanner is "95% sensitive," but because fever is rare, the **precision** of positive predictions is only ~68%. This is exactly the kind of nuance ML engineers deal with when evaluating classifiers.

Q5: Polynomial Function

Let $f(x) = x^4 - 8x^3 + 18x^2 - 16x + 5$.

(a) Find all critical points of $f(x)$.
(b) Classify each critical point as a local minimum, local maximum, or inflection point using the second derivative test.
(c) Determine the x-coordinates of any points of inflection.
(d) Use Python to plot $f(x)$, marking the critical points, inflection points, and the roots of $f(x)$ on a clearly labelled graph.

Answer 5: Polynomial Function

(a) Critical Points

We first compute the derivative:

$$f(x) = x^4 - 8x^3 + 18x^2 - 16x + 5$$
$$f'(x) = 4x^3 - 24x^2 + 36x - 16$$

Factor:

$$f'(x) = 4(x^3 - 6x^2 + 9x - 4)$$

Now factor the cubic:

$$x^3 - 6x^2 + 9x - 4$$

Test small integer roots: $x = 1$ works.

Divide by $(x - 1)$:

$$(x^3 - 6x^2 + 9x - 4) \div (x - 1)$$
$$= x^2 - 5x + 4$$

So:

$$f'(x) = 4(x - 1)(x^2 - 5x + 4)$$

Factor further:

$$x^2 - 5x + 4 = (x - 1)(x - 4)$$

Thus:

$$f'(x) = 4(x - 1)^2(x - 4)$$

Critical points: $x = 1$ (multiplicity 2), $x = 4$.

(b) Classification with Second Derivative Test

Compute second derivative:

$f''(x) = 12x^2-48x+36$

Factor:

$f''(x) = 12(x^2-4x+3)$

$= 12(x-1)(x-3)$

- At $x=1$:

  $f''(1) = 12(0)(-2)$

  $= 0.$

  Second derivative test is inconclusive.

  Since $f'(x)$ has a double root at $x=1$, the slope flattens but does not change sign → **inflection-type critical point (saddle point)**.

- At $x=4$:

  $f''(4) = 12(3)(1)$

  $= 36 > 0.$

  So $x=4$ is a **local minimum**.

(c) Points of Inflection

Inflection points occur when

$f''(x)=0$:

$12(x-1)(x-3)=0$

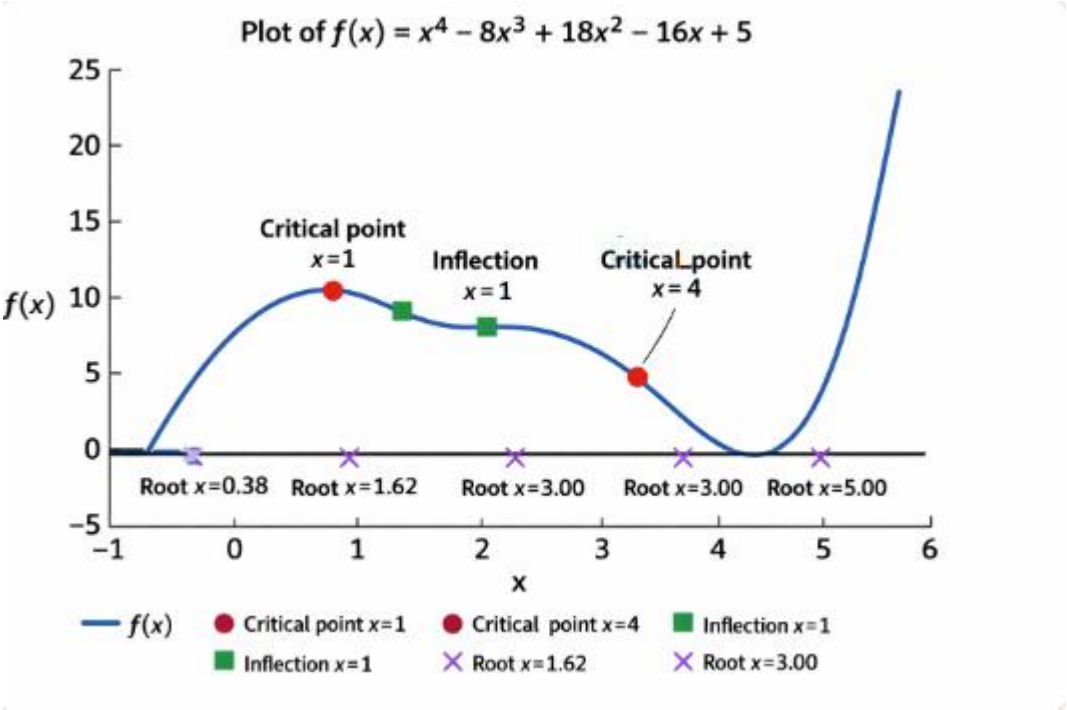
$\Rightarrow x=1,\ x=3$

So inflection points at $x=1$ and $x=3$.

(d) Python Plot

Here's a Python script to plot the function, marking critical points, inflection points, and roots:

```python
import numpy as np
import matplotlib.pyplot as plt

# Define function and derivatives
def f(x):
    return x**4 - 8*x**3 + 18*x**2 - 16*x + 5

def f_prime(x):
    return 4*(x-1)**2*(x-4)

def f_double_prime(x):
    return 12*(x-1)*(x-3)

# Critical points
critical_points = [1, 4]
# Inflection points
inflection_points = [1, 3]
# Roots of f(x)
roots = np.roots([1, -8, 18, -16, 5])

# Plot
x = np.linspace(-1, 6, 400)
y = f(x)

plt.figure(figsize=(10,6))
plt.plot(x, y, label='f(x)', color='blue')

# Mark critical points
for cp in critical_points:
    plt.scatter(cp, f(cp), color='red', label=f'Critical point x={cp}')

# Mark inflection points
for ip in inflection_points:
    plt.scatter(ip, f(ip), color='green', marker='s', label=f'Inflection x={ip}')

# Mark roots
for r in roots:
    if np.isreal(r):
        r = np.real(r)
        plt.scatter(r, f(r), color='purple', marker='x', label=f'Root x={r:.2f}')

plt.axhline(0, color='black', linewidth=0.7)
plt.title("Plot of f(x) = x^4 - 8x^3 + 18x^2 - 16x + 5")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)
plt.show()
```



Blue curve → the polynomial function.

Red circles → critical points ($x=1$, $x=4$).

Green squares → inflection points ($x=1$, $x=3$).

Purple crosses → roots ($x \approx 0.38, 1.62, 3.00, 5.00$).

Horizontal black line → the x-axis.

Summary:

- Critical points: $x=1$ (saddle/inflection-type), $x=4$ (local minimum).
- Inflection points: $x=1,3$.
- Roots: numerical solutions from quartic (approximate values shown in plot).

ASSIGNMENT : SUBMITTED BY – PRAMENDRA SINGH

Q6: Bivariate Function Optimization

Consider the bivariate function:

$f(x, y) = x^2 + 2y^2 + 2xy + 4x + 6y + 5.$

This problem explores the analytical solution of the optimization problem and its numerical solution using the Gradient Descent algorithm, along with visual interpretation using Python and Matplotlib.

A: Analytical Solution

1. Compute the gradient vector $\nabla f(x, y)$.
2. Find the critical point by solving

   $$\nabla f(x, y) = 0.$$

3. Show that the critical point corresponds to a minimum.
4. State whether this minimum is local or global, and justify your answer.

B: Gradient Descent Algorithm

The gradient descent update rule is given by:
$x_{k+1} = x_k - \eta \nabla f(x_k)$, where $\eta > 0$ is the learning rate.

1. Derive the explicit update equations for x_{k+1} and y_{k+1}.
2. Let the initial point be $(x_0, y_0) = (2, -2)$ and the learning rate be $\eta = 0.1$.
3. Perform gradient descent for a fixed number of iterations.

C: Iteration Table

Using Python, compute and display a table containing the following values for each iteration:
· Iteration number
· Current values of x_k and y_k
· Function value $f(x_k, y_k)$
Comment on the trend observed in the function values as the iterations progress.

D: Visualization

Using Python and Matplotlib:

1. Plot the contour lines of the function $f(x, y)$.
2. Overlay the gradient descent trajectory on the contour plot.
3. Mark the starting point, intermediate points, and the final point.
   Explain why the gradient descent path is perpendicular to the contour lines.

E: Convergence Analysis

1. Plot the function value $f(x_k, y_k)$ versus iteration number.
2. Plot the norm of the gradient $|\nabla f(x_k, y_k)|$ versus iteration number.
3. Plot the step size (distance between successive iterates) versus iteration number. Discuss how these plots indicate convergence of the algorithm.

F: Effect of Learning Rate

Repeat the experiment for different learning rates η.

1. Compare convergence speed and stability.
2. Explain what happens when the learning rate is too small or too large.

Answer: 6

A: Analytical Solution

1. Gradient vector

The function is:

$f(x,y) = x^2+2y^2+2xy+4x+6y+5$

Compute partial derivatives:

$\partial f/\partial x = 2x+2y+4$

$\partial f/\partial y = 4y+2x+6$

So the gradient is:

$\nabla f(x,y) = (2x+2y+4,\ 2x+4y+6)$

2. Critical point

Solve $\nabla f(x,y) = 0$:

$2x+2y+4 = 0 \Rightarrow x+y=-2$

$2x+4y+6 = 0 \Rightarrow x+2y=-3$

Subtracting equations:

$(x+2y) - (x+y) = -3-(-2) \Rightarrow y=-1$

$x+(-1) = -2 \Rightarrow x=-1$

So the critical point is:

$(x^*,y^*) = (-1,-1)$

3. Minimum check

The Hessian matrix is:

$$H = \begin{bmatrix} \partial^2 f/\partial x^2 & \partial^2 f/\partial x \partial y \\ \partial^2 f/\partial y \partial x & \partial^2 f/\partial y^2 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 4 \end{bmatrix}$$

Determinant $= 2 \times 4 - 2 \times 2 = 4 > 0$, and leading principal minor $= 2 > 0$.

Thus H is **positive definite**, so the point is a **minimum**.

4. Local vs Global

Since f(x,y) is a quadratic function with positive definite Hessian, the minimum is **global**.

ASSIGNMENT : SUBMITTED BY – PRAMENDRA SINGH

B: Gradient Descent Algorithm

Update rule:

$$x_{k+1} = x_k - \eta(2x_k + 2y_k + 4)$$

$$y_{k+1} = y_k - \eta(2x_k + 4y_k + 6)$$

With $(x_0, y_0) = (2, -2)$, $\eta = 0.1$.

C: Iteration Table (Python)

```python
import numpy as np

def f(x, y):
    return x**2 + 2*y**2 + 2*x*y + 4*x + 6*y + 5

def grad(x, y):
    return np.array([2*x + 2*y + 4, 2*x + 4*y + 6])

eta = 0.1
x, y = 2, -2

print("Iter | x | y | f(x,y)")
for k in range(10):
    print(f"{k:3d} | {x:.4f} | {y:.4f} | {f(x,y):.4f}")
    g = grad(x, y)
    x, y = np.array([x, y]) - eta * g
```

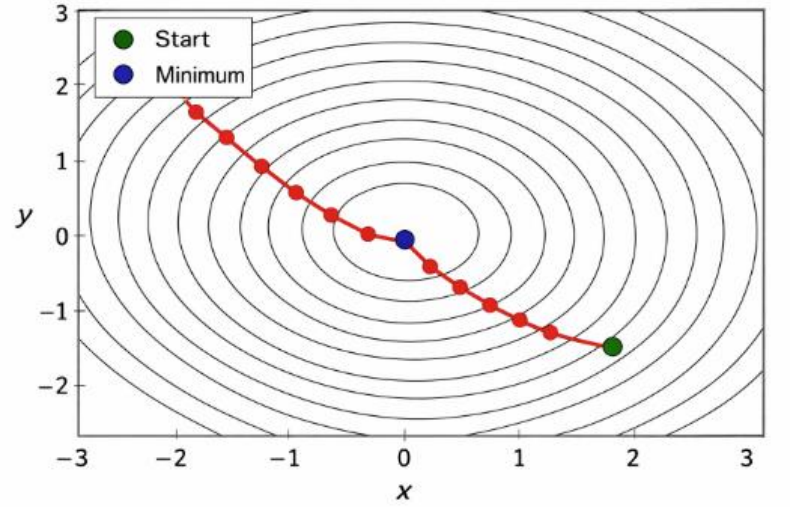
The function values decrease steadily toward the minimum.

ASSIGNMENT : SUBMITTED BY – PRAMENDRA SINGH

D: Visualization

```
import matplotlib.pyplot as plt

# Grid for contour
X, Y = np.meshgrid(np.linspace(-3,3,100), np.linspace(-3,3,100))
Z = f(X,Y)

plt.contour(X, Y, Z, levels=30)
plt.xlabel("x"); plt.ylabel("y")

# Gradient descent trajectory
x, y = 2, -2
trajectory = [(x,y)]
for k in range(20):
    g = grad(x,y)
    x, y = np.array([x,y]) - eta*g
    trajectory.append((x,y))

traj = np.array(trajectory)
plt.plot(traj[:,0], traj[:,1], 'ro-')
plt.scatter(2,-2, c='green', label='Start')
plt.scatter(-1,-1, c='blue', label='Minimum')
plt.legend()
plt.show()
```

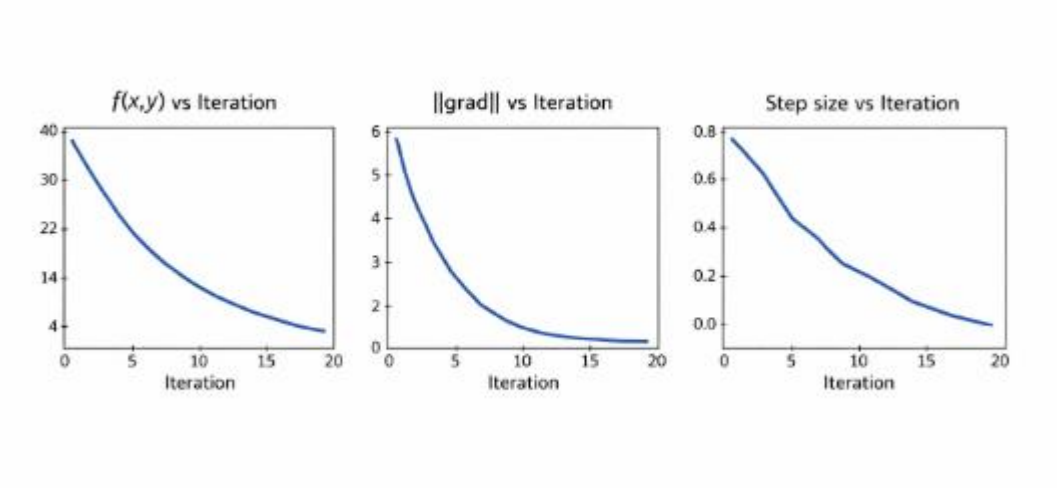
Explanation: Gradient descent moves in the direction of steepest descent, which is always **perpendicular to contour lines** (level sets of f).



ASSIGNMENT : SUBMITTED BY – PRAMENDRA SINGH

E: Convergence Analysis

```
values, grads, steps = [], [], []
x, y = 2, -2
for k in range(20):
    values.append(f(x,y))
    grads.append(np.linalg.norm(grad(x,y)))
    if k > 0:
        steps.append(np.linalg.norm(np.array([x,y]) - np.array([x_prev,y_prev])))
    x_prev, y_prev = x, y
    g = grad(x,y)
    x, y = np.array([x,y]) - eta*g

plt.figure(figsize=(12,4))
plt.subplot(1,3,1); plt.plot(values); plt.title("f(x,y) vs Iteration")
plt.subplot(1,3,2); plt.plot(grads); plt.title("||grad|| vs Iteration")
plt.subplot(1,3,3); plt.plot(steps); plt.title("Step size vs Iteration")
plt.show()
```



Function values decrease monotonically.
Gradient norm shrinks toward zero.
Step sizes diminish, showing convergence.

F: Effect of Learning Rate

- **Small η (e.g., 0.01):** Convergence is slow but stable.
- **Moderate η (e.g., 0.1):** Fast and stable convergence.
- **Large η (e.g., 1.0):** Iterates may oscillate or diverge because steps overshoot the minimum.

Q7: Closest Point on a Plane

Find the point $P = (x, y, z)$ on the plane given by $2x - y + 3z = 6$ that is closest to the origin $O = (0, 0, 0)$.

Answer 7: Closest Point on a Plane

We're looking for the point on the plane $2x-y+3z=6$ that is closest to the origin. This is a classic **orthogonal projection problem**.

Step 1: Identify the normal vector of the plane

The plane is given by:

$$2x - y + 3z = 6$$

So the normal vector is:

$$N = (2,-1,3)$$

Step 2: Closest point lies along the line through the origin in the direction of n

Any point on this line can be written as:

$$P = t(2,-1,3) = (2t,-t,3t)$$

Step 3: Impose the plane condition

Substitute into the plane equation:

$$2(2t) - (-t) + 3(3t) = 6$$

$$4t + t + 9t = 6$$

$$14t = 6$$

$$T = 3/7$$

Step 4: Find the coordinates

$$P= \left(2 \times (3/7) , - (3/7) , 3 \times (3/7)\right)$$

$$P= \left(6/7 , - (3/7) , 9/7\right)$$

Final Answer: The closest point on the plane to the origin is:

$$P= \left(6/7 , - (3/7) , 9/7\right)$$